

RAG & Embeddings

Teach an LLM about *your* documents: load, chunk, embed, store in a vector database, and retrieve the right passages to answer questions — with embeddings from both **Ollama** and the **Gemini API**.

THIS VOLUME COVERS

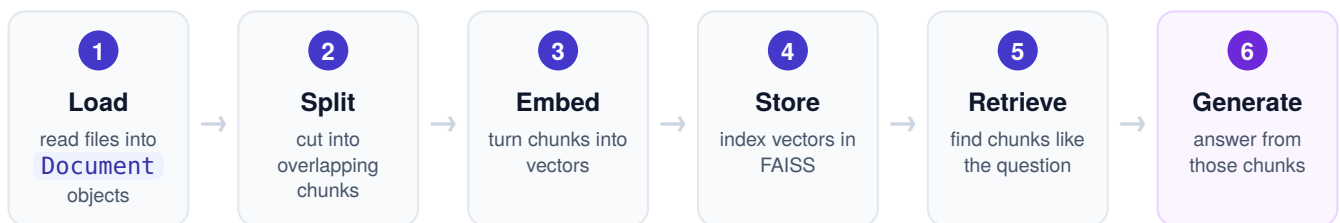
Phase 3 — RAG & Embeddings
Project: chat with a PDF

Why this is the part that matters

A bare LLM only knows what was in its training data. It has never seen your company handbook, last week's report, or the PDF on your desk — and when asked about them, it tends to guess confidently and wrongly. **Retrieval-Augmented Generation (RAG)** fixes this.

The idea is simple and powerful: before the model answers, your application *retrieves* the most relevant passages from your own documents and pastes them into the prompt as context. The model then answers **grounded in those passages** instead of from memory. This is how “chat with your documents” products work, and it is where LangChain earns its keep.

THE RAG PIPELINE IN ONE PICTURE



Steps 1–4 happen once, when you *index* your documents. Steps 5–6 happen every time someone asks a question. This part walks through each step, then assembles them into a working “chat with a PDF” application.

THE ENGINE TOGGLE COVERS TWO MODELS NOW

RAG uses a **chat model** (to write the answer) *and* an **embedding model** (to turn text into vectors). “Choose your engine” now sets both.

```
PYTHON

from dotenv import load_dotenv
load_dotenv()

# ---- Option A: everything local with Ollama ----
from langchain_ollama import ChatOllama, OllamaEmbeddings
llm = ChatOllama(model="llama3.1", temperature=0)
embeddings = OllamaEmbeddings(model="nomic-embed-text")

# ---- Option B: everything cloud with Gemini ----
# from langchain_google_genai import ChatGoogleGenerativeAI, GoogleGenerativeAIEmbeddings
# llm = ChatGoogleGenerativeAI(model="gemini-2.5-flash", temperature=0)
# embeddings = GoogleGenerativeAIEmbeddings(model="gemini-embedding-001")
```

PHASE 3

RAG & Embeddings

We will demystify embeddings (the maths that lets a computer judge “these two sentences mean similar things”), learn to chunk documents sensibly, store the vectors in the free, local **FAISS** database, and build the retrieval chain that turns all of it into grounded answers.

3.1 Embeddings: turning meaning into numbers

An **embedding** is a list of numbers (a *vector*) that represents the meaning of a piece of text. An embedding model is trained so that texts with similar meaning land close together in this numeric space, and unrelated texts land far apart. “How do I get a refund?” and “What is your returns policy?” produce nearby vectors even though they share almost no words. That is what makes **semantic search** possible.

Both engines expose an embedding model with the same tiny interface:

 PYTHON (OLLAMA)

```
# Ollama (local; pull it first with: ollama pull nomic-embed-text)
from langchain_ollama import OllamaEmbeddings
embeddings = OllamaEmbeddings(model="nomic-embed-text")

vector = embeddings.embed_query("How do I get a refund?")
print(len(vector))      # the dimensionality, e.g. 768 numbers
print(vector[:5])       # [-0.012, 0.044, -0.001, ...]
```

 PYTHON (GEMINI)

```
# Gemini (cloud; needs GOOGLE_API_KEY)
from langchain_google_genai import GoogleGenerativeAIEmbeddings
embeddings = GoogleGenerativeAIEmbeddings(model="gemini-embedding-001")

vector = embeddings.embed_query("How do I get a refund?")
print(len(vector))      # Gemini embeddings are higher-dimensional
print(vector[:5])
```

You rarely call `embed_query` yourself in practice — the vector store does it for you. But seeing the raw vector makes the concept concrete: a chunk of text becomes a fixed-length list of floats, and “relevant” means “its vector is close to the question’s vector.”

Pick one embedding model and stick with it

The vectors from different models are not comparable, and a vector store is built around one model’s output. If you re-index with Ollama embeddings you must also query with Ollama embeddings. Mixing `nomic-embed-text` and Gemini in one index will return nonsense.

3.2 Chunking: cutting documents to the right size

You cannot embed a whole 50-page PDF as one vector — the meaning would be hopelessly averaged out, and it would not fit a model's limits. Instead you **split** the text into smaller **chunks** (a few paragraphs each). Retrieval then returns just the handful of chunks that actually answer the question, keeping the prompt focused and cheap.

The workhorse splitter is `RecursiveCharacterTextSplitter`. It tries to break on natural boundaries first (paragraphs, then lines, then sentences) so chunks stay coherent. Two settings matter: `chunk_size` (roughly how big each chunk is) and `chunk_overlap` (how much text neighbouring chunks share, so a sentence split across a boundary is not lost).



TERMINAL

```
pip install -U langchain-text-splitters
```



PYTHON

```
from langchain_text_splitters import RecursiveCharacterTextSplitter
from langchain_community.document_loaders import PyPDFLoader

docs = PyPDFLoader("handbook.pdf").load()    # from Part 2: List of Documents

splitter = RecursiveCharacterTextSplitter(
    chunk_size=1000,          # ~1000 characters per chunk
    chunk_overlap=200,        # neighbours share 200 characters
)
chunks = splitter.split_documents(docs)

print(len(chunks), "chunks")
print(chunks[0].page_content[:120])
print(chunks[0].metadata)    # keeps source + page number
```



Sensible starting numbers

For prose and reports, `chunk_size=1000` with `chunk_overlap=200` is a reliable default. Smaller chunks give more precise retrieval but more pieces to manage; larger chunks give more context per hit but blur relevance. Tune only after the basic pipeline works.

3.3 The vector store: FAISS

A **vector store** embeds every chunk, stores the vectors, and — given a new query vector — finds the closest ones fast. We use **FAISS** (Facebook AI Similarity Search): it is free, runs entirely on your machine, needs no server, and is the perfect choice for learning and for small-to-mid projects.

TERMINAL

```
pip install -U langchain-community faiss-cpu
```

PYTHON

```
from langchain_community.vectorstores import FAISS

# Build the index: this embeds every chunk with your chosen model
vectorstore = FAISS.from_documents(chunks, embeddings)

# Quick semantic search (no LLM yet) -- returns the closest chunks
hits = vectorstore.similarity_search("how do refunds work?", k=3)
for d in hits:
    print("-", d.page_content[:80])
```

For real use you wrap the store in a **retriever** — a Runnable (remember Part 2) that takes a question string and returns the top matching documents, so it slots straight into an LCEL chain:

PYTHON

```
retriever = vectorstore.as_retriever(search_kwargs={"k": 4})
docs = retriever.invoke("how do refunds work?") # it's a Runnable
```

Indexing can be slow, so save the index to disk and reload it instead of rebuilding every run:

PYTHON

```
vectorstore.save_local("faiss_index") # write once

vectorstore = FAISS.load_local(
    "faiss_index",
    embeddings, # same model used to build it
    allow_dangerous_deserialization=True, # required to load a local index
)
```

! Why the scary-sounding flag?

FAISS loads its index with Python's pickle format, which can execute code, so LangChain makes you opt in with `allow_dangerous_deserialization=True`. It is safe when *you* created the index file. Never load an index file from an untrusted source.

3.4 The retrieval chain (LCEL)

Now we connect retrieval to generation using exactly the LCEL helpers from Part 2. The shape is: take the question, run it through the retriever to get context, fill a prompt with both, send it to the model, and parse the text. A tiny helper joins the retrieved documents into one string.

```
PYTHON

from langchain_core.prompts import ChatPromptTemplate
from langchain_core.output_parsers import StrOutputParser
from langchain_core.runnables import RunnablePassthrough

def format_docs(docs):
    return "\n\n".join(d.page_content for d in docs)

prompt = ChatPromptTemplate.from_template(
    "Answer the question using ONLY the context below. "
    "If the answer is not in the context, say you don't know.\n\n"
    "Context:\n{context}\n\nQuestion: {question}"
)

rag_chain = (
    {"context": retriever | format_docs, "question": RunnablePassthrough()}
    | prompt
    | llm
    | StrOutputParser()
)

print(rag_chain.invoke("How do I get a refund?"))
```

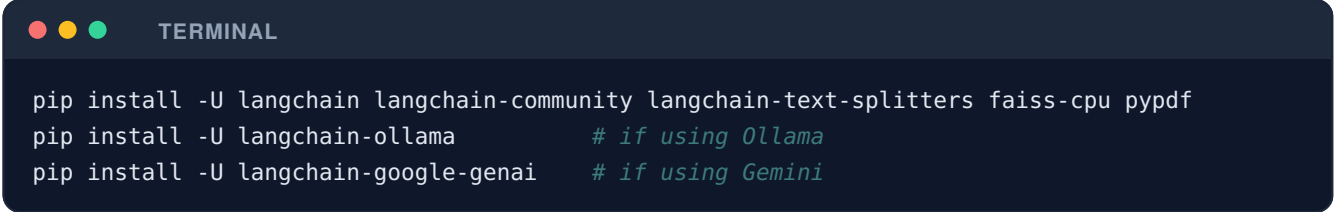
Read the dict at the top as a `RunnableParallel` (Part 2): the question string flows into the retriever to build `context`, and `RunnablePassthrough()` forwards the same string unchanged as `question`. Both keys land in the prompt. Because the whole thing is a chain, it also supports `stream()` for a live, typed-out answer — which the project uses.

* Grounding beats guessing

The instruction “use ONLY the context... otherwise say you don’t know” is what turns a confident hallucinator into a trustworthy assistant. It tells the model to defer to your documents rather than its memory — the single most important line in a RAG prompt.

Build a “chat with a PDF” app

This single script is the whole phase end to end: it loads a PDF, splits it, embeds the chunks with your chosen engine, indexes them in FAISS, and then runs a question-and-answer loop that retrieves and answers — streaming the reply. Point it at any PDF (a manual, a paper, a contract) and start asking.

A terminal window with a dark background and a title bar containing three colored circles (red, yellow, green) and the word "TERMINAL". The terminal displays three pip install commands for installing the necessary packages.

```
pip install -U langchain langchain-community langchain-text-splitters faiss-cpu pypdf
pip install -U langchain-ollama           # if using Ollama
pip install -U langchain-google-genai     # if using Gemini
```




```
# chat_with_pdf.py
import sys
from dotenv import load_dotenv
from langchain_community.document_loaders import PyPDFLoader
from langchain_text_splitters import RecursiveCharacterTextSplitter
from langchain_community.vectorstores import FAISS
from langchain_core.prompts import ChatPromptTemplate
from langchain_core.output_parsers import StrOutputParser
from langchain_core.runnables import RunnablePassthrough

load_dotenv()

# --- choose your engine (chat model + embedding model) -----
from langchain_ollama import ChatOllama, OllamaEmbeddings
llm = ChatOllama(model="llama3.1", temperature=0)
embeddings = OllamaEmbeddings(model="nomic-embed-text")

# from langchain_google_genai import ChatGoogleGenerativeAI, GoogleGenerativeAIEmbeddings
# llm = ChatGoogleGenerativeAI(model="gemini-2.5-flash", temperature=0)
# embeddings = GoogleGenerativeAIEmbeddings(model="gemini-embedding-001")
# -----

pdf_path = sys.argv[1] if len(sys.argv) > 1 else "handbook.pdf"

print(f"Indexing {pdf_path} ...")
docs = PyPDFLoader(pdf_path).load()
chunks = RecursiveCharacterTextSplitter(
    chunk_size=1000, chunk_overlap=200
).split_documents(docs)
vectorstore = FAISS.from_documents(chunks, embeddings)
retriever = vectorstore.as_retriever(search_kwargs={"k": 4})
print(f"Indexed {len(chunks)} chunks. Ask away!\n")

prompt = ChatPromptTemplate.from_template(
    "Answer the question using ONLY the context below. "
    "If the answer is not in the context, say you don't know.\n\n"
    "Context:\n{context}\n\nQuestion: {question}"
)

def format_docs(docs):
    return "\n\n".join(d.page_content for d in docs)

rag_chain = (
    {"context": retriever | format_docs, "question": RunnablePassthrough()}
    | prompt | llm | StrOutputParser()
)

while True:
    question = input("you > ").strip()
    if question.lower() in {"quit", "exit"}:
        break
```

```
if not question:
    continue
print("bot > ", end="", flush=True)
for chunk in rag_chain.stream(question):
    print(chunk, end="", flush=True)
print("\n")
```

Run it against a PDF and interrogate it:

● ● ● TERMINAL

```
python chat_with_pdf.py handbook.pdf
```

● ● ● OUTPUT

```
Indexing handbook.pdf ...
```

```
Indexed 87 chunks. Ask away!
```

```
you > How many vacation days do new employees get?
```

```
bot > New employees receive 15 paid vacation days in their first year,
increasing to 20 days after two years of service.
```

```
you > What's the capital of France?
```

```
bot > I don't know - that isn't covered in the provided document.
```

```
you > quit
```

Make it your own

(1) Flip to the Gemini engine and re-run — note you must re-index, since the embeddings differ. (2) Cache the index with `save_local` / `load_local` so restarts are instant. (3) Print the source page for each answer by returning the retrieved documents' `metadata` alongside the text. Each touches one Phase 3 idea.

Part 3 recap & what comes next

You can now make an LLM answer from your own documents instead of guessing — the core skill behind most real LLM products.

You can now

- Explain what an embedding is and why it enables semantic search.
- Chunk documents with sensible size and overlap.
- Build, search, save, and reload a FAISS index.
- Wrap a store as a retriever and drop it into an LCEL chain.
- Ground answers in retrieved context to stop hallucination.

Key objects introduced

- `OllamaEmbeddings` , `GoogleGenerativeAIEmbeddings`
- `RecursiveCharacterTextSplitter`
- `FAISS` · `from_documents` , `as_retriever`
- `save_local` / `load_local`
- the `{context, question} | prompt | llm | parser` chain

★ Coming in Part 4 — Agents & Tools

So far the model only talks. Next it **acts**. We use the `@tool` decorator to write custom tools (a calculator, a search), then build an **agent** that decides on its own which tool to call and when — using the modern `create_agent` API, which runs on LangGraph. The project is a **web-search agent**, shown for both Ollama and Gemini.

End of Part 3 · The LangChain Practitioner's Guide